

دليل تغييرات الـ API للفرونتند

دليل تغييرات الـ API للفرونتند

نسخة: بعد المراحل 1 → 6 (تأمين + Caching/Logging + Indexes + Sync + Ledger + Bootstrap) آخر تحديث: 22-05-2026

ملخص سريع — TL;DR

الأولوية	التغيير	الـ Endpoints المتأثرة
حرج	المبالغ المالية بقت <code>string</code> بدل <code>number</code>	كل endpoints الديون والفواتير والتقارير
عالي	<code>POST /api/sync/push</code> يرفض <code>overpay</code> بـ 400	<code>/api/sync/push</code>
عالي	شكل <code>GET /api/health</code> body اتغير	<code>/api/health</code>
متوسط	max مقبول على 100 Pagination	كل endpoints list
متوسط	جديد (429) Rate limiting	<code>/auth/*</code>
متوسط	موّدد غير <code>Error shape</code> <code>PrismaExceptionFilter</code>	كل endpoints بـ DB
منخفض	Cross-tenant references → 403	<code>/api/sync/push</code>
منخفض	على <code>?force-fresh=true</code> <code>/api/sync/init</code>	<code>/api/sync/init</code>
منخفض	حذف العميل بقي <code>soft-delete</code>	<code>DELETE /api/customers/:id</code>

ملاحظات عامة: - **✗** ما تم حذف ولا endpoint - **✗** ما تغير اسم ولا URL - **✗** ما تغيرت طرق المصادقة (نفس الـ JWT Bearer token)

1. المبالغ المالية بقت Strings بدل Numbers

السبب

كنا نستخدم JavaScript Numbers مع `toFixed(2)` للحساب المالي. ده بييسبب float drift:

```
0.1 + 0.2 === 0.30000000000000004 // ❌
```

بعد التحديث: السيرفر يحسب بـ `Prisma.Decimal` (دقة عشرية مطلقة) ويرجع القيم كـ strings لتفادي فقدان الدقة في JS.

كيف الفرونت يتعامل معاها

للعرض: حوّل لـ `Number` عند الـ rendering فقط:

```
// قبل
<span>{invoice.total}</span>

// بعد
<span>{Number(invoice.total).toFixed(2)}</span>

// أو مع تنسيق عربي:
<span>{new Intl.NumberFormat('ar-EG').format(Number(invoice.total))}</span>
```

لحسابات على الفرونت: استخدم `decimal.js` أو `big.js` (ممش JS Number):

```
import Decimal from 'decimal.js';

const total = new Decimal(invoice.total);
const tax = total.times(0.15);
const grandTotal = total.plus(tax);
display(grandTotal.toFixed(2));
```

للإرسال للسيرفر: ابعت `number` عادي (السيرفر يحوّل لـ `Decimal` تلقائياً):

```
fetch('/api/debts/123/pay', {
  method: 'POST',
  body: JSON.stringify({ amount: 250.50 }) // number صحيح
});
```

قائمة الحقول المتأثرة بالتفصيل

`POST /api/debts/:id/pay`

```
{
  "payment": {
    "id": "uuid",
    - "amount": 60,
    + "amount": "60",
    "date": "2026-05-22T...",
    "notes": null,
    "debtId": "uuid"
  },
  "debt": {
    "id": "uuid",
    - "paid": 60,
    + "paid": "60",
    - "remaining": 40,
    + "remaining": "40",
    "isPaid": false
  }
}
```

POST /api/debts/customer/:customerId/pay

```
{
  "customer": { ... },
  - "paymentApplied": 100,
  + "paymentApplied": "100",
  "affectedDebts": [
    - { "debtId": "uuid", "amountPaid": 60, "isPaid": false }
    + { "debtId": "uuid", "amountPaid": "60", "isPaid": false }
  ],
  "summary": {
    "totalDebts": 3,
    "unpaidCount": 1,
    - "totalAmount": 500,
    - "totalPaid": 460,
    - "totalRemaining": 40,
    + "totalAmount": "500",
    + "totalPaid": "460",
    + "totalRemaining": "40"
  }
}
```

GET /api/debts

كل عنصر في `data[]` فيه `amount` , `paid` , `remaining` (الديسيمالات في ال Prisma model أصلاً مرجوعة ك strings).

GET /api/debts/summary

```
{
  "totalDebts": 5,
- "totalAmount": 1500,
- "totalPaid": 800,
- "totalRemaining": 700,
+ "totalAmount": "1500",
+ "totalPaid": "800",
+ "totalRemaining": "700",
  "unpaidCount": 2,
- "unpaidRemaining": 700
+ "unpaidRemaining": "700"
}
```

GET /api/debts/customer/:customerId

```
{
  "customer": { ... },
  "summary": {
    "totalDebts": 3,
    "unpaidCount": 1,
-   "totalAmount": 500,
-   "totalPaid": 460,
-   "totalRemaining": 40,
+   "totalAmount": "500",
+   "totalPaid": "460",
+   "totalRemaining": "40"
  },
  "debts": [ /* amount/paid/remaining strings as well */ ]
}
```

GET /api/customers/:id/debt-summary

```
{
  "customer": { ... },
  "summary": {
-   "totalDebt": 200,
-   "totalPaid": 150,
-   "totalRemaining": 50,
+   "totalDebt": "200",
+   "totalPaid": "150",
+   "totalRemaining": "50",
    "unpaidCount": 1,
    "totalDebts": 2
  }
}
```

/by-number/:n + /:id + GET /api/invoices + PATCH /api/invoices/:id + POST /api/invoices

strings → remaining , paid , total •
strings → items[].total , items[].unitCost , items[].price •

GET /api/invoices/daily-sales

```
{
  "date": "2026-05-22",
  "summary": {
    "invoiceCount": 15,
    - "totalSales": 1200,
    - "totalPaid": 900,
    - "totalCash": 500,
    - "totalOnline": 400,
    - "totalDebt": 300,
    + "totalSales": "1200",
    + "totalPaid": "900",
    + "totalCash": "500",
    + "totalOnline": "400",
    + "totalDebt": "300"
  },
  "invoices": [ /* total/paid/remaining strings */ ]
}
```

الحقول اللي ما تغيّرتش

النوع	الأمثلة
UUIDs	id
Boolean	isPaid , isActive , isDeleted
Integer	number , stock , quantity (رقم الفاتورة), counts
Enum	paymentMethod , role , status
Date strings	date , createdAt , updatedAt
Plain text	name , phone , notes

2. POST /api/sync/push — رفض الـ Overpayment

قبل

لو دفعة (debt payment) قيمتها أكبر من المتبقي على الدين، السيرفر كان يخصمها بصمت للمتبقي ويرجع 200.

بعد

السيرفر يرفض ال payload كله بـ 400 ويرجّع تفاصيل واضحة:

```
{
  "statusCode": 400,
  "error": "Bad Request",
  "message": "المبلغ (100) يتجاوز المتبقي على الدين (50). أعد المزامنة بعد f2a-...: الدفعه 3
  جلب آخر الحالة"
}
```

التعامل المطلوب في الفرونت

```
try {
  const res = await api.post('/api/sync/push', payload);
} catch (err) {
  if (err.status === 400 && err.body.message.includes('يتجاوز المتبقي')) {
    // اعرض للكاشير: "بياناتك قديمة، جاري التحديث 1."
    showToast('بياناتك قديمة، جاري التحديث');

    // 2. من السيرفر اجلب state اطلب من السيرفر
    const fresh = await api.get('/api/sync/init?force-fresh=true');
    db.replaceWith(fresh);

    // 3. اطلب من المستخدم إعادة المحاولة
    promptUserToRetry();
  }
}
```

السبب

الـ silent capping كان يخفي bugs على الفرونت (بيانات stale) ويسبب ال ledger يفترق عن واقع الكاشير.

3. GET /api/health — جديد + Shape 503

قبل

```
{ "status": "ok" }
```

كان دائماً يرجّع 200 حتى لو ال DB واقعة.

بعد (مع [@nestjs/terminus](https://github.com/nestjs/terminus))

عندما ال DB متاحة (200 status):

```
{
  "status": "ok",
  "info": {
    "database": { "status": "up" }
  },
  "error": {},
  "details": {
    "database": { "status": "up" }
  }
}
```

عندما ال DB واقعة (status 503):

```
{
  "status": "error",
  "info": {},
  "error": {
    "database": {
      "status": "down",
      "message": "..."
    }
  },
  "details": {
    "database": { "status": "down", "message": "..." }
  }
}
```

التعامل

```
const isHealthy = res.ok && res.body.status === 'ok';
// أو
const dbStatus = res.body.details?.database?.status; // 'up' | 'down'
```

4. max مقفولة على 100 Pagination

كل ال endpoints (/customers , /products , /invoices , /debts , ...) لو الفرونت بعث `?limit=999` :- ❌ ما هيرجع 999
 - ✅ هيرجع 100 عنصر فقط - ✅ ال `meta.limit` في ال response هيقول 100 (مش 999)

```
GET /api/customers?page=1&limit=99999
```

```
{
  "data": [ /* عناصر بس 100 */,
  "meta": {
    "total": 5234,
    "page": 1,
    "limit": 100,      // المُطبَّق فعلاً ←
    "totalPages": 53
  }
}
```

التعامل

لو محتاج تنزيل البيانات كلها (مثلاً export), الفرونت لازم يـ paginate:

```
async function fetchAll(endpoint: string) {
  const all = [];
  let page = 1;
  while (true) {
    const res = await api.get(`${endpoint}?page=${page}&limit=100`);
    all.push(...res.data);
    if (page ≥ res.meta.totalPages) break;
    page++;
  }
  return all;
}
```

5. 🟡 Rate Limiting (429)

في حدود صارمة على endpoints المصادقة:

نافذة	الحد	Endpoint
كل دقيقة	5 محاولات	POST /api/auth/login
دقيقة	5	POST /api/auth/super-admin/login
ساعة	5	POST /api/auth/register
15 دقيقة	5	POST /api/auth/verify-email
ساعة	3	POST /api/auth/forgot-password
15 دقيقة	5	POST /api/auth/reset-password
دقيقة	200 طلب	باقي endpoints

عند الوصول للحد:

HTTP/1.1 429 Too Many Requests

```
{
  "statusCode": 429,
  "message": "ThrottlerException: Too Many Requests"
}
```

التعامل في الفرونت

```
if (response.status === 429) {
  // لو موجود (بالتواني) اقرأ Retry-After header
  const retryAfter = response.headers.get('retry-after');
  showError(`ثانية ${retryAfter || 60} تجاوزت عدد المحاولات. حاول بعد`);
  disableSubmitButtonFor(retryAfter * 1000);
}
```

6. ● Error Response Shape مودّ

كل أخطاء قاعدة البيانات بقت تطلع بنفس الشكل عبر `PrismaExceptionFilter` :

Prisma code	HTTP Status	ال Error
P2002	409 Conflict	سجل مكرر (unique violation)
P2025	404 Not Found	السجل غير موجود
P2003	400 Bad Request	مرجع غير صالح (FK violation)
P2000	400 Bad Request	القيمة طويلة جداً
P2014	400 Bad Request	علاقة مطلوبة انتهكة
أي خطأ DB آخر	500	"حدث خطأ في قاعدة البيانات"

مثال

```
HTTP/1.1 409 Conflict

{
  "statusCode": 409,
  "error": "Conflict",
  "message": "(email) سجل مكرر: القيمة موجودة مسبقاً",
  "code": "P2002",
  "target": "email"
}
```

التعامل

```
function handleApiError(err: ApiError) {
  // message بدل ما تفسّر الـ `code` اعتمد على
  if (err.code === 'P2002') {
    showFieldError(err.target, 'القيمة دي مستخدمة قبل كده');
  } else if (err.code === 'P2025') {
    showError('السجل المطلوب غير موجود');
  } else {
    showError(err.message);
  }
}
```

قبل التحديث

كانت ترجع 500 مع stack trace كامل (مشكلة أمنية). الآن لا stack traces بتظهر للـ client.

7. Cross-tenant References → 403

في `POST /api/sync/push` ، لو ال payload فيه `customerId` أو `productId` أو `invoiceId` تابع لمتجر تاني:

```
HTTP/1.1 403 Forbidden
```

```
{
  "statusCode": 403,
  "message": "لا ينتمي إلى متجرك payload أحد العملاء في الـ",
  "error": "Forbidden"
}
```

كان بيرجع 500 (FK error) أو حتى ينجح أحياناً.

التعامل

لو شُفت 403 على `push`، يعني ال IndexedDB المحلي عنده بيانات بايئة من تسجيل دخول قديم. اعمل `/sync/init?force-fresh=true` لإعادة seeding من الصفر.

8. `?force-fresh=true` على `/sync/init`

`/sync/init` بقى cached لـ 30 ثانية في الذاكرة على السيرفر. لو الفرونت محتاج يضمن أحدث بيانات (مثلاً بعد رفض من `/push` أو بعد ما الكاشير عدّل منتج)، يقدر يتجاوز ال cache:

```
GET /api/sync/init?force-fresh=true
```

أو:

```
GET /api/sync/init?force-fresh=1
```

متى تستخدمها

- بعد كل 400 من `/sync/push` (إجباري)
- زر "تحديث يدوي" في الواجهة
- بعد التحديثات الإدارية الكبيرة

متى لا تستخدمها

- على كل تحميل للتطبيق (يكسر فايذة ال cache)

9. حذف العميل (Soft Delete)

DELETE /api/customers/:id لسه يرجع **No Content 204** كما كان. لكن داخلياً:

- ✓ العميل ما يتمحيش من الـ DB
- ✓ يتأرشف (يتعلم `isDeleted = true`)
- ✓ يختفي من القوائم (`/api/customers` , `/api/sync/init`)
- ✓ فواتيره وديونه القديمة بتفضل ظاهرة في سجل المتجر باسمه

السلوك للفرونت

نفس السلوك المرئي بالضبط — العميل يختفي. لكن لو الفرونت كان يفترض إن `id` العميل بيتحرّر للاستخدام مرة ثانية، ده مش صحيح بعد التحديث (الـ `id` يفضل مأخوذ).

السبب

كان فيه bug في برودكشن: حذف عميل عنده ديون (حتى لو مدفوعة) كان يفشل بـ `FK constraint error`. الـ `soft delete` يحل المشكلة.

10. تغييرات صامتة في رسائل الأخطاء

بعض رسائل الأخطاء اتغيرت لتعطي تفاصيل أوضح. الفرونت لو معتمد على نص الرسالة الحرفي للـ `branching`, يحتاج تحديث.

أمثلة

- بيع منتج وهو خلاص: `"الكمية المطلوبة (3) من \"اسم المنتج\" تتجاوز المخزون المتوفر (1) "` (كان أحياناً بيرجع رسائل أبسط)
- منتج معطل: `"غير متوفر أو معطل \"X\" المنتج "` (جديد)

التوصية

استخدم HTTP status codes و `body.code` للـ `branching`, مش نص الـ `message`.

ملحق A — قائمة كاملة بكل الـ Endpoints (للتأكيد، URLs ما تغيّرتش)

Auth

- `POST /api/auth/register` — تسجيل متجر جديد
- `POST /api/auth/login` — دخول مستخدم
- `POST /api/auth/super-admin/login` — دخول `SUPER_ADMIN`
- `POST /api/auth/verify-email` — تأكيد البريد بـ OTP
- `POST /api/auth/forgot-password` — طلب reset link

- — استخدام الـ reset token — `POST /api/auth/reset-password`

Customers

- — إنشاء عميل — `POST /api/customers`
- — قائمة العملاء — `GET /api/customers?page&limit&search`
- — تفاصيل العميل + ديونه + فواتيره — `GET /api/customers/:id`
- — ملخص ديون العميل — `GET /api/customers/:id/debt-summary`
- — تحديث — `PATCH /api/customers/:id`
- — أرشفة (soft-delete) — `DELETE /api/customers/:id`

Products

- — إنشاء منتج — `POST /api/products`
- — قائمة — `GET /api/products?page&limit&search&isActive`
- — منتجات قاربت على النفاد — `GET /api/products/low-stock`
- — بحث بالباركود (مع 5 cache دقائق) — `GET /api/products/barcode/:barcode`
- — تفاصيل المنتج — `GET /api/products/:id`
- — تحديث — `PATCH /api/products/:id`
- — حذف نهائي — `DELETE /api/products/:id`

Invoices

- — إنشاء فاتورة — `POST /api/invoices`
- — قائمة — `GET /api/invoices?page&limit&search&paymentMethod&dateFrom&dateTo`
- — مبيعات يوم — `GET /api/invoices/daily-sales?date=YYYY-MM-DD`
- — تفاصيل الفاتورة — `GET /api/invoices/:id`
- — بحث برقم الفاتورة — `GET /api/invoices/by-number/:n`
- — تحديث — `PATCH /api/invoices/:id`
- — حذف (مع استعادة المخزون) — `DELETE /api/invoices/:id`

Debts

- — قائمة — `GET /api/debts?page&limit&search&customerId&isPaid&dateFrom&dateTo`
- — ملخص الديون للمتجر — `GET /api/debts/summary`
- — كل ديون عميل — `GET /api/debts/customer/:customerId`
- — تفاصيل دين — `GET /api/debts/:id`
- — دفعات الدين — `GET /api/debts/:id/payments`
- — دفعة على دين محدد — `POST /api/debts/:id/pay`
- — دفعة موزعة على ديون عميل — `POST /api/debts/customer/:customerId/pay`
- — إلغاء دفعة (ADMIN) — `DELETE /api/debts/:debtId/payments/:paymentId`

Sync (PWA)

- بيانات التهيئة الأولية — `GET /api/sync/init?force-fresh=true|1`
- رفع البيانات المنشأة أو فلاتين — `POST /api/sync/push`

Reports

- الأرباح اليومية — `GET /api/reports/daily-profit?date=YYYY-MM-DD`

Health

- فحص صحة السيرفر (لا يحتاج auth) — `GET /api/health`

Backup (Admin)

- (موجودة قبل وبعد، لم تتغير)

Store / User (Admin)

- (موجودة قبل وبعد، لم تتغير)

ملحق B — حدود ال (Phase 4) Sync Push

`POST /api/sync/push` بقي عنده حدود صارمة لمنع DoS:

الحقل	الحد الأقصى
<code>invoices.length</code>	200 فاتورة في ال payload الواحد
<code>debts.length</code>	500 دين
<code>debtPayments.length</code>	1000 دفعة
<code>invoices[].items.length</code>	100 بند في الفاتورة
حجم ال payload (إجمالي)	2MB

تجاوز الحد → 400 مع رسالة عربية واضحة.

Auth Header (لم يتغير)

```
Authorization: Bearer <jwt-token>
```

CORS

السيفر بيقبل من `ALLOWED_ORIGINS` فقط (متغير بيئة). تأكد دومين الفرونت موجود في ال `allowlist`.

Security Headers (عبر Helmet)

- `X-Content-Type-Options: nosniff`
 - (HSTS) `Strict-Transport-Security`
 - كل response دلوقتي فيه: `Content-Security-Policy` (default) - `X-Frame-Options: SAMEORIGIN`
- ما حدش من دول بيأثر على ال `requests`، بس بيحمي من XSS و clickjacking.

التواصل

لو في endpoint محدد محتاج توضيح أكثر، أو في حالة edge case معينة مش متغطية هنا، ابعت سؤال محدد بـ: - ال URL - شكل ال request body - شكل ال response body المتوقع vs المستلم
ها يتم تحديث ال Markdown ده مع كل phase جديدة.